

TzakGPT: Architecture and Implementation of a Context-Aware Agentic CLI

Technical Architecture Review

June 17, 2026

Abstract

TzakGPT is a local, terminal-based AI assistant engineered to provide an interactive, agentic workflow over a single language model API (DeepSeek). Unlike standard conversational wrappers, TzakGPT is imbued with file system awareness, autonomous file editing capabilities with visual diffs, secure shell execution, and persistent session memory. This paper details the system architecture, dissects the modular codebase, and analyzes the design principles that enable robust human-in-the-loop LLM interactions directly within a developer's working directory.

Contents

1	Introduction	2
2	System Architecture	2
2.1	Execution Orchestration (<code>main.py</code>)	2
2.2	LLM Integration (<code>clients.py</code>)	2
2.3	The Agentic Soul (<code>soul.py</code>)	2
2.4	Tool Execution (<code>tools.py</code>)	3
2.5	State and Telemetry (<code>session.py</code>)	3
2.6	User Interface (<code>display.py</code>)	3
3	Prompt Engineering and Behavioral Guardrails	3
4	Conclusion	3

1 Introduction

As Large Language Models (LLMs) evolve from simple text generators to autonomous agents, the need for robust, transparent, and controllable interfaces becomes paramount. TzakGPT addresses this need by bridging the gap between a remote LLM API and the local developer environment. Built in Python, it leverages a single application loop to orchestrate prompt construction, tool execution, state management, and rich terminal user interfaces (TUIs).

The core philosophy of TzakGPT rests on three pillars:

1. **Contextual Awareness:** The model is injected with real-time directory states and session activity logs.
2. **Human-in-the-Loop Safety:** High-risk operations, such as shell commands, require explicit user confirmation.
3. **Verifiable Actions:** File modifications are immediately parsed and presented as color-coded unified diffs, ensuring transparency.

2 System Architecture

The system is divided into six highly cohesive, loosely coupled modules. This separation of concerns ensures maintainability and scalability.

2.1 Execution Orchestration (`main.py`)

The `main.py` module serves as the central nervous system. It orchestrates the primary “agent loop,” which is capable of executing up to 10 sequential tool-call iterations per user prompt. It handles user inputs using `prompt_toolkit`, allowing for interactive, auto-completing slash commands (e.g., `/help`, `/clear`, `/save`). Responses are rendered asynchronously using the `rich` library for markdown parsing and syntax highlighting.

2.2 LLM Integration (`clients.py`)

All external communication is localized to `clients.py`. Using the OpenAI-compatible Python SDK, it interfaces with the DeepSeek API. This module abstracts the API request/response cycle, returning a standardized tuple containing the response kind (text or tool call), the message object, and the token usage statistics.

2.3 The Agentic Soul (`soul.py`)

The `soul.py` module manages the model’s identity, constraints, and memory context. It constructs the payload sent to the LLM by combining:

- The fixed system prompt dictating narration and verification rules.
- A dynamically generated map of the current working directory.
- A sliding window of conversation history.

To mitigate token exhaustion, a sliding window algorithm triggers when the user turn count exceeds 10. Oldest messages are sent back to the LLM for summarization, collapsing context into a single dense memory string while preserving the 6 most recent interactions.

2.4 Tool Execution (`tools.py`)

The model interacts with the local system via strictly defined Python functions:

- `read_file(path)`: Ingests file contents for analysis.
- `write_file(path, content)`: Overwrites files and generates a unified diff against the previous state using Python’s `difflib`.
- `run_command(cmd)`: Executes shell commands via `subprocess` with a 60-second timeout to prevent blocking.
- `list_directory(path)`: Returns structured JSON of files and subdirectories.

2.5 State and Telemetry (`session.py`)

Session continuity is maintained by tracking every tool invocation and state change. It logs timestamps, action types, and execution success states. Additionally, it tracks cumulative token usage (input/output) across the session, triggering warnings when context thresholds (50,000 tokens) are approached.

2.6 User Interface (`display.py`)

Visual feedback is crucial for a CLI agent. `display.py` encapsulates all rich console interactions. It provides a visual confirmation prompt mapping to ‘Y’ (Run), ‘N’ (Skip), or ‘E’ (Edit), and parses unified diff outputs into line-numbered, color-coded `rich.panel.Panel` components.

3 Prompt Engineering and Behavioral Guardrails

The reliability of TzakGPT hinges heavily on its system prompt. The model is explicitly instructed to:

“Before starting any task that requires more than one tool call, state the plan in one or two sentences... After writing a file, always call `read_file` on the same path to verify the content.”

This chain-of-thought and self-verification pattern drastically reduces the hallucination of completed tasks. If an action results in an error string, the model is instructed to halt the loop and surface the error to the user rather than failing silently.

4 Conclusion

TzakGPT demonstrates how standard LLM APIs can be transformed into highly capable, localized agentic tools. By combining a rich terminal interface with strict tool-calling schemas and human-in-the-loop safety mechanisms, it provides a powerful, transparent coding assistant that operates safely within the boundaries of the local file system.